

Lecture 12

Bufferization: Tensor -> MemRef, Alias, and Lifetime Analysis

NPU compiler boundary where abstract tensor SSA becomes physical buffers

지난 강의 : Dialect Conversion 으로 legal op set 을 만든다

이번 강의 : tensor value semantics 를 memref storage semantics 로 변환

다음 강의 : MemRef layout, memory space, SRAM/DRAM mapping

Bufferization 이 왜 late-stage boundary 인지 설명한다

One-Shot Bufferize 의 analysis/rewrite 구조를 이해한다

DPS, outs operand, in-place bufferization 조건을 읽는다

NPU memory planning 관점의 alias/lifetime audit 를 수행한다

NPU 성능은 compute 보다 data movement 에 자주 제한된다
불필요한 tensor copy 는 SRAM bandwidth 와 DRAM traffic 을 동시에 증가시킨다
잘못된 in-place 결정은 output corruption 으로 이어진다
runtime ABI 는 결국 memref/tensor descriptor 와 command buffer 를 요구한다

tensor: immutable SSA value,
shape/type 중심, high-level
optimization 에 유리

memref: mutable buffer reference,
layout/memory space/lifetime 이
명시됨

bufferization 은 추상 값에서 물리
storage obligation 으로 넘어가는 경계

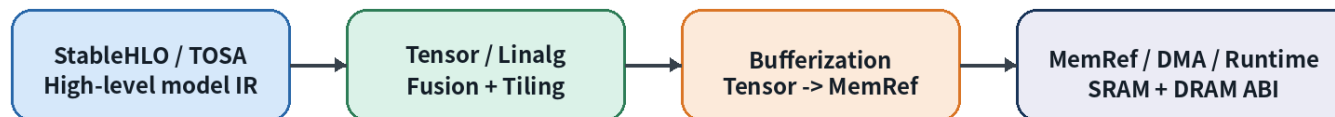
StableHLO/TOSA -> Tensor/Linalg
단계에서 fuse/tile/canonicalize

Bufferization 이후에는
alloc/copy/load/store/dealloc
비용이 드러남

NPU compiler 는 bufferization
전후의 pass boundary 를 명확히
관리해야 함

Bufferization Boundary in an NPU Compiler

Do high-level graph/tensor transforms before committing to physical buffers.



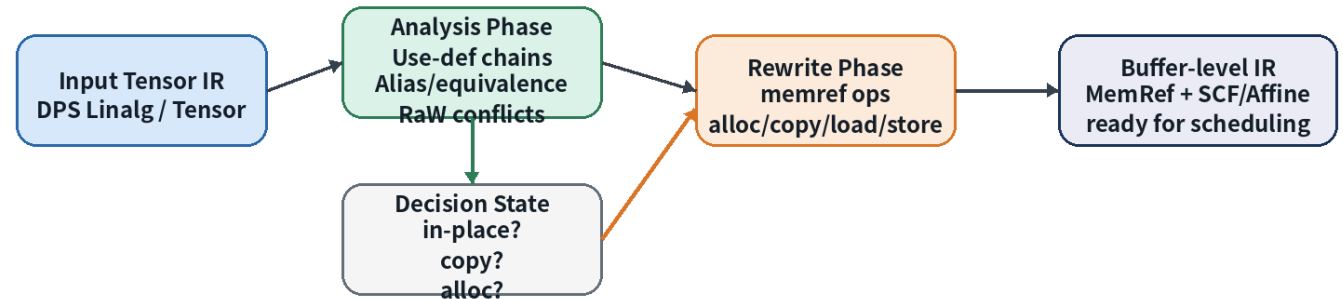
MLIR bufferization 은 tensor semantics op 를 memref semantics op 로 바꾸는 과정

One-Shot Bufferize 는 tensor IR 을 memref IR 로 rewrite 하는 핵심 pass
많은 변환은 tensor land 에서 더 쉽기 때문에 bufferization 은 pipeline 후반에 배치

whole-function analysis 로 tensor
SSA use-def chain 을 분석
alias/equivalence set 과 RaW
conflict 를 고려해 in-place 여부 결정
analysis phase 후 rewrite phase
에서 memref IR 생성
BufferizableOpInterface 를 구현한
op 가 대상

One-Shot Bufferize: Analysis then Rewrite

The best NPU result comes from making buffer decisions with tensor-level SSA information still available.



DPS op 는 result 마다 대응되는 destination operand 를 가진다
linalg.generic/linalg.matmul 의 outs operand 가 대표 예
conflict 가 없다면 result buffer 는 destination buffer 를 reuse 가능
NPU 에서는 outs 가 accumulator/output tile buffer 후보가 된다

destination buffer 가 아직 필요한 값을 덮어쓰지 않아야 함
후속 read 가 있으면 RaW conflict 를 고려해야 함
shape/layout/memory space 가 compatible 해야 함
aliasing 이 불명확하면 copy 가 안전한 fallback

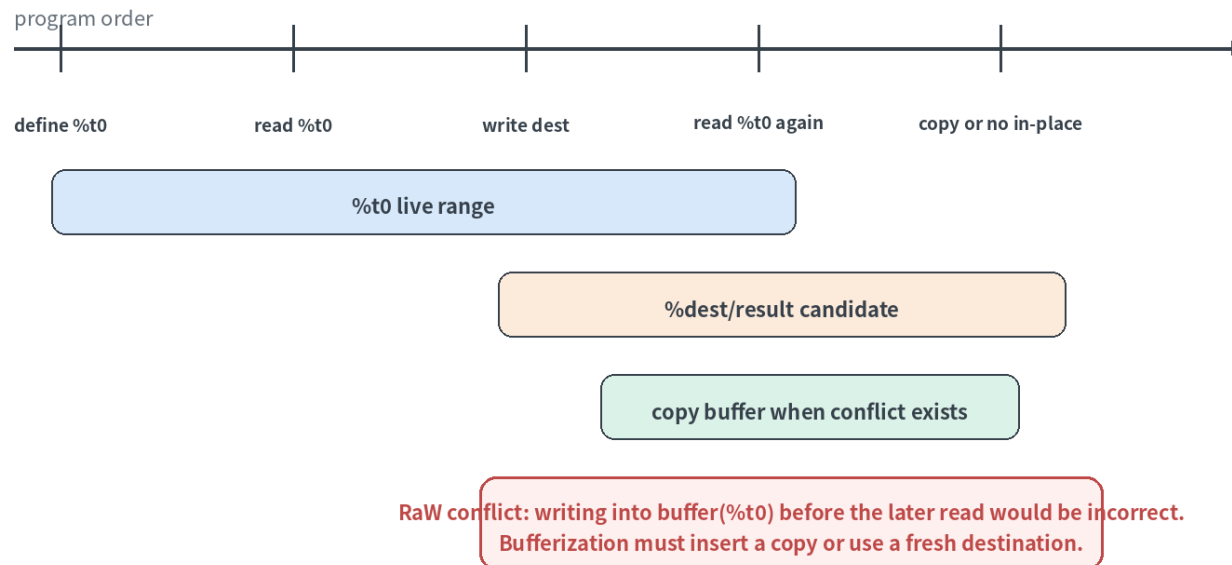
Read-after-Write conflict: 나중에 읽을 값을 먼저 덮어쓰는 문제

compiler 는 SSA use-def chain 에서 conflict 를 감지

conflict 시 새 alloc 또는 copy 를 삽입

NPU 에서는 SRAM scratch reuse 정책과 직접 연결

Alias / Lifetime Audit for NPU Memory Planning

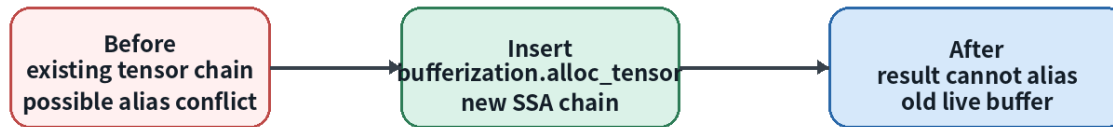


fresh tensor SSA chain 을 만드는
anchor

bufferized result 는 다른 buffer 와
alias 하지 않는 fresh allocation 으로
취급

RaW conflict 를 깨는 데 유용
단, 과도한 사용은 memory
footprint 증가

bufferization.alloc_tensor as a Fresh-Buffer Anchor



Use it sparingly: every fresh allocation may increase SRAM/DRAM pressure, but it can break a correctness conflict.

bufferization.to_tensor: 기존 memref 를 tensor world 로 가져온다
bufferization.to_buffer: tensor 의 future buffer 를 memref 로 노출한다
materialize_in_destination: 특정 buffer 에 tensor materialization 요구
restrict/writable 같은 alias guarantee 가 correctness 에 중요

기본적으로 function boundary bufferization 은 별도 정책이 필요
tensor arg/result 를 memref arg/result 로 바꿀지 결정
runtime ABI 와 연결되는 지점
NPU compiler 에서는 model entry/exit descriptor contract 와 맞춰야 함

unknown-type-conversion 이 layout map 생성 정책에 영향
fully dynamic layout 은 partial bufferization composition 에 안전
identity layout 은 단순하지만 cast/copy 증가 가능
NPU 에서는 layout propagation 과 bufferization 순서가 중요

tensor.empty -> memref.alloc 또는 bufferization alloc 계열
tensor.extract/insert -> memref.load/store 또는 copy/subview
linalg-on-tensors -> linalg-on-buffers
deallocation/ownership pass 가 memory leak 방지에 필요

DRAM: large persistent tensors /
model IO

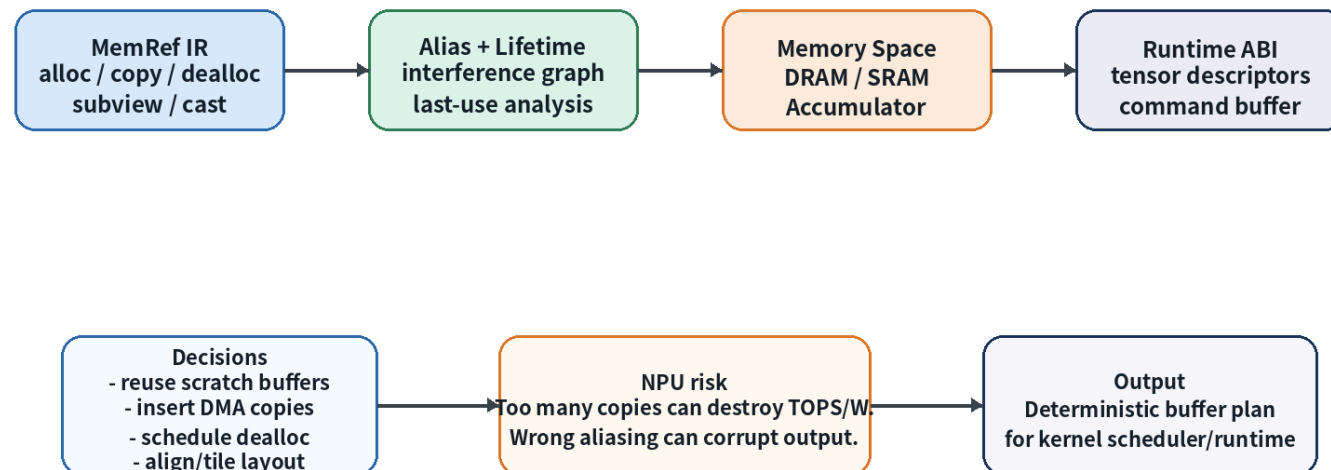
SRAM: tile buffers, double
buffering, intermediate activation

Accumulator: wider partial sums,
reduction buffers

memory_space attribute 는 target-
specific contract 로 설계

From Bufferization to NPU Memory Planning

Bufferization is the compiler boundary where abstract tensors become physical storage obligations.



같은 physical buffer 를 공유하는 SSA value set 을 추적
subview/extract_slice/insert_slice 는 alias region 을 만든다
parallel DMA/compute schedule 에서는 alias conflict 가 더 위험
memory reuse 는 lifetime non-overlap 이 증명될 때만 가능

definition 부터 last use 까지 live range 를 잡는다
live range overlap 이 있으면 같은 scratch buffer reuse 불가
tile loop 내부 lifetime 은 loop-carried value 와 yield 도 고려
runtime buffer pool 설계의 compiler-side 근거가 된다

Before: `tensor.empty` + `linalg.matmul` + `linalg.generic` epilogue

Bufferization: C output tensor 가 memref C buffer 로 materialize

Fusion 이 된 상태라면 intermediate tensor copy 를 줄일 수 있음

After: `linalg-on-buffers` 또는 `npu_kernel.matmul_epilogue` 로 lowering 가능

mean/variance reduction result lifetime 이 짧음
normalized activation 은 gamma/beta broadcast 와 alias 하면 안 됨
reduction buffer 와 activation buffer memory space 가 다를 수 있음
bufferization 전 fusion/legalization 이 매우 중요

IR 에서 memref.copy, bufferization.clone, alloc_tensor 증가를 추적
one-shot-bufferize debug flags 로 copy insertion 원인 확인
FileCheck 로 불필요 copy 가 생기지 않는지 회귀 테스트
NPU 에서는 copy count 를 bandwidth model 과 연결

canonicalize/cse

tensor/linalg fusion and tiling

one-shot-bufferize

buffer deallocation / ownership

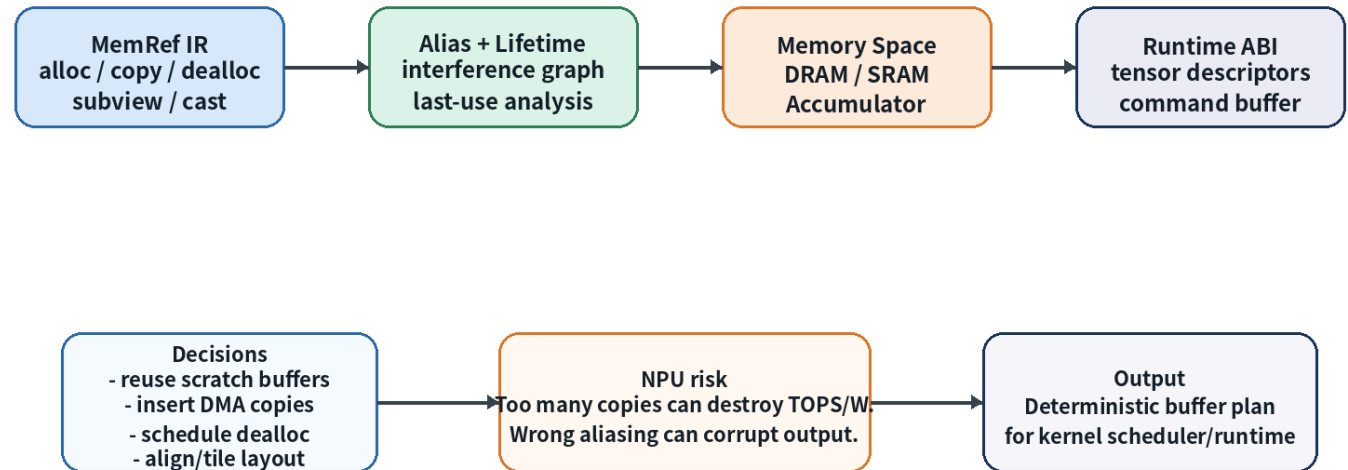
memref layout/memory-space assignment

DMA/barrier/runtime lowering

StableHLO/TOSA legalize
Linalg-on-tensors fusion/tiling
insert tensor.empty / alloc_tensor
anchors intentionally
one-shot-bufferize with required
dialect models
buffer-level cleanup, memory
planning, command emission

From Bufferization to NPU Memory Planning

Bufferization is the compiler boundary where abstract tensors become physical storage obligations.



All tensor ops are bufferizable or
boundary ops are intentional
No unexpected RaW conflicts
remain
Function boundary policy matches
runtime ABI

Copies are expected and justified
Memory spaces are preserved
through lowering

Peak live bytes fits SRAM budget?
Copy bytes per tile acceptable?
DRAM load/store coalescing
preserved?

Double buffering creates no alias
hazard?
Can scratch buffers be reused across
non-overlapping live ranges?

CHECK-LABEL for function boundary

CHECK-NOT: memref.copy for expected in-place case

CHECK: memref.alloc for expected fresh buffer case

negative test for unsupported tensor op

keep tests semantic, not overfitted to SSA names

Annotate tensor IR and mark destination operands

For each result, write candidate buffer(result)

Identify possible in-place decisions

Mark any follow-up read that can cause RaW conflict

Run/inspect a pseudo one-shot-bufferize pipeline

Compare before/after IR

Count alloc/copy/dealloc operations

Explain every copy in one sentence

Design NPU buffer lifetime table

Map buffers to DRAM/SRAM/ACCUM

Decide which scratch buffers can share storage

Write runtime tensor descriptor fields

Bufferizing before fusion and creating unnecessary intermediates

Ignoring function boundary ABI

Treating memory_space as a cosmetic attribute

Assuming subviews do not alias

Using alloc_tensor everywhere instead of fixing the producer chain

For MatMul+Bias+ReLU, produce before/after bufferization IR sketch
Create a memory lifetime chart for 3 tile buffers and 1 accumulator
Write FileCheck test that rejects unexpected memref.copy
Document runtime ABI change caused by tensor->memref boundary

Bufferization is a correctness and performance boundary

One-Shot Bufferize uses SSA use-def analysis to make in-place/copy decisions

DPS style is the best input form for high-quality bufferization

NPU compiler should connect bufferization output to memory planning and runtime ABI

MLIR Bufferization

MLIR Bufferization Dialect

MLIR MemRef Dialect

MLIR Builtin Dialect - MemRef type and memory space

MLIR Tensor Dialect

MLIR Linalg Dialect